

S2.01. Nociones básicas SQL

Cristina Navarrete
Data Analytics Curso 26-27
IT Academy

NIVELL1

EXERCICI 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Creamos la base de datos **transactions**:

```
6      -- Creamos la base de datos
7      CREATE DATABASE IF NOT EXISTS transactions;
8
9
10
11
12
13
14
15
16
17
18
19
20
```

Output

#	Time	Action	Message	Duration / Fetch
5114	13:45:22	CREATE DATABASE IF NOT EXISTS transactions	1 row(s) affected	0.016 sec

Creamos la tabla **company**:

```
12      -- Creamos la tabla company
13      CREATE TABLE IF NOT EXISTS company (
14          id VARCHAR(15) PRIMARY KEY,
15          company_name VARCHAR(255),
16          phone VARCHAR(15),
17          email VARCHAR(100),
18          country VARCHAR(100),
19          website VARCHAR(255)
20      );
21
```

Output

#	Time	Action	Message	Duration / Fetch
5116	13:46:40	USE transactions	0 row(s) affected	0.000 sec
5117	13:46:45	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KE...	0 row(s) affected	0.032 sec

Creamos la tabla **transaction**:

```
24      CREATE TABLE IF NOT EXISTS transaction (
25          id VARCHAR(255) PRIMARY KEY,
26          credit_card_id VARCHAR(15) REFERENCES credit_card(id),
27          company_id VARCHAR(20),
28          user_id INT REFERENCES user(id),
29          lat FLOAT,
30          longitude FLOAT,
31          timestamp TIMESTAMP,
32          amount DECIMAL(10, 2),
33          declined BOOLEAN,
34          FOREIGN KEY (company_id) REFERENCES company(id)
35      );
36
```

Output

#	Time	Action	Message	Duration / Fetch
5117	13:46:45	CREATE TABLE IF NOT EXISTS company (id VARCHAR(15) PRIMARY KE...	0 row(s) affected	0.032 sec
5118	13:47:48	CREATE TABLE IF NOT EXISTS transaction (id VARCHAR(255) PRIMARY ...	0 row(s) affected	0.062 sec

Insertamos los registros de la tabla **company** y **transaction** de el archivo **SQL N1-Ex.1** **dades_introduir**:

```

1 • USE transactions;
2
3 -- Insertamos datos de company
4 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
5 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
6 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
7 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
8 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
9 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
10 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
11 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
12 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
13 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
14 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
15 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
16 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
17 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
18 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (
19 • INSERT INTO company (id, company_name, phone, email, country, website) VALUES (

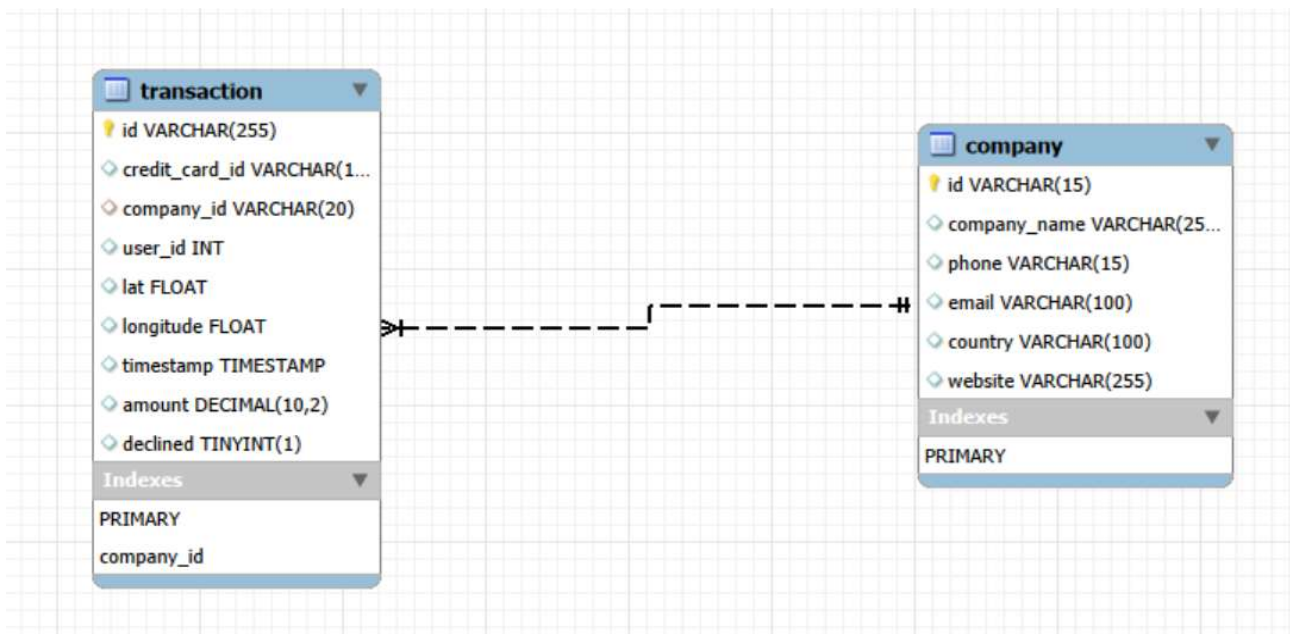
```

'b-2222', 'Ac Fermentum Incorporated', '06 85 56 52'
'b-2226', 'Magna A Neque Industries', '04 14 44 64 6'
'b-2230', 'Fusce Corp.', '08 14 97 58 85', 'risus@pr'
'b-2234', 'Convallis In Incorporated', '06 66 57 29'
'b-2238', 'Ante Iaculis Nec Foundation', '08 23 04 9'
'b-2242', 'Donec Ltd', '01 25 51 37 37', 'at.iaculis'
'b-2246', 'Sed Nunc Ltd', '02 62 64 73 48', 'nibh@ya'
'b-2250', 'Amet Nulla Donec Corporation', '07 15 25'
'b-2254', 'Nascetur Ridiculus Mus Inc.', '06 26 87 6'
'b-2258', 'Vestibulum Lorem PC', '02 02 87 33 40', '
'b-2262', 'Gravida Sagittis LLP', '03 81 28 33 97', '
'b-2266', 'Mus Aenean Eget Foundation', '06 25 15 52'
'b-2270', 'Dis Parturient Institute', '05 36 29 78 7'
'b-2274', 'Sed LLC', '01 63 16 26 52', 'at@outlook.c'
'b-2278', 'Arcu LLP', '06 46 04 41 45', 'dui@aol.ca'
'b-2282', 'Pretium Neque Corp.', '07 77 48 55 28', '

Output

#	Time	Action	Message	Duration / Fetch
105218	14:00:55	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, ti...	1 row(s) affected	0.000 sec
105219	14:00:55	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, ti...	1 row(s) affected	0.000 sec

Diagrama:



Estamos trabajando con la base de datos de una empresa de venta de productos en línea. Su nombre es **transactions**.

La base de datos está formada por las tablas **company** y **transaction**.

- La tabla **company** contiene información sobre las empresas: 'id', 'company_name', 'phone', 'email', 'country', 'website', etc.
- La tabla **transaction** contiene información sobre las transacciones realizadas: id, credit_card_id, company_id, user_id, timestamp, amount, declined, etc.
- En cada tabla, id es la Primary Key, que identifica de forma única cada registro.
- El diagrama muestra una relación **1:N (uno a muchos)** entre las tablas **company** y **transaction**, donde **company** es la tabla del lado 1 y **transaction** es la tabla del lado N (muchos). Esto significa que una empresa puede estar asociada a muchas transacciones, mientras que cada transacción pertenece a una única empresa. La relación se establece mediante 'company_id' de **transaction**, que es la clave foránea que referencia al 'id' de **company**.

EXERCICI 2

Utilitzant JOIN realitzaràs les següents consultes:

- *Llistat dels països que estan generant vendes.*

The screenshot displays a database query interface. At the top, a SQL query is entered in a text area:

```
45 • SELECT company.country
46 FROM transaction
47 JOIN company
48 ON transaction.company_id = company.id
49 WHERE transaction.declined = 0
50 GROUP BY company.country;
```

Below the query, the 'Result Grid' shows the results of the query. The grid has a single column labeled 'country'. The results are listed as follows:

country
Netherlands
Sweden
Ireland
United States
Belgium
Canada
Germany
Norway
France

At the bottom, the 'Output' section shows the execution log. It includes the following entries:

#	Time	Action	Message	Duration / Fetch
37	22:53:31	SELECT company.id, company.company_name, ROUND(AVG(transaction.amount))...	1 row(s) returned	0.406 sec / 0.000 sec
38	22:55:02	SELECT company.country FROM transaction JOIN company ON transaction.comp...	16 row(s) returned	0.266 sec / 0.000 sec

- Des de quants països es generen les vendes:

```

54 • SELECT COUNT(DISTINCT company.country)
55 FROM transaction
56 JOIN company
57 ON company.id = transaction.company_id
58 WHERE transaction.declined = 0;

```

Result Grid

COUNT(DISTINCT company.country)
15

Result 18 × Read Only

Output

#	Time	Action	Message	Duration / Fetch
✓ 38	22:55:02	SELECT company.country FROM transaction JOIN company ON transaction.comp...	16 row(s) returned	0.266 sec / 0.000 sec
✓ 39	22:55:57	SELECT COUNT(DISTINCT company.country) FROM transaction JOIN company ...	1 row(s) returned	0.265 sec / 0.000 sec

- Identifica la companyia amb la mitjana més gran de vendes:

```

59 • SELECT company.id, company.company_name, ROUND(AVG(transaction.amount),2) AS media_ventas
60 FROM transaction
61 JOIN company
62 ON transaction.company_id = company.id
63 WHERE transaction.declined = 0
64 GROUP BY company.id, company.company_name
65 ORDER BY media_ventas DESC
66 LIMIT 1;
67
RR

```

Result Grid

id	company_name	media_ventas
b-2222	Ac Fermentum Incorporated	284.91

Result 16 × Read Only

Output

#	Time	Action	Message	Duration / Fetch
✗ 36	22:53:23	SELECT company.id, company.company_name, ROUND(AVG(transaction.amount))...	Error Code: 1146. Table 'online_sales.transaction' doesn't exist	0.000 sec
✓ 37	22:53:31	SELECT company.id, company.company_name, ROUND(AVG(transaction.amount))...	1 row(s) returned	0.406 sec / 0.000 sec

EXERCICI 3

Utilitzant només subconsultes (sense utilitzar JOIN):

- Mostra totes les transaccions realitzades per empreses d'Alemanya

```
70 • SELECT *
71 FROM transaction
72 WHERE EXISTS (SELECT company.id
73               FROM company
74               WHERE country = 'Germany');
75
```

Result Grid

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00043A49-2949-494B-A5DD-A5BAE3BB19DD	CcS-9294	b-2458	4713	46.1999	1.43554	2024-08-28 07:16:46	395.43	0
000447FE-8650-4DCF-85DE-C7ED0EE1CAAD	CcS-5019	b-2370	438	41.5972	12.2218	2016-12-21 20:07:18	155.63	0
00045D68-ED2E-4F2F-8186-CEE074D875D0	CcS-6699	b-2390	2118	29.7573	-95.3796	2020-07-14 15:37:45	326.01	0
000481C3-1C26-4FEF-83A0-4CD0EB004BBD	CcS-6696	b-2230	2115	53.5489	-113.503	2017-09-04 19:44:53	161.60	0
00051AA4-9CBE-4268-B070-C38062A1B3E2	CcS-7606	b-2266	3025	52.2084	5.69081	2017-01-05 18:19:25	148.91	0
0008A312-EDFE-4A4F-BC99-E9C92EC3CA4D	CcU-3358	b-2598	215	53.5535	-113.499	2023-09-23 04:51:43	294.59	0
0009A151-9BCE-4F31-9053-A468FF77FΔΔR	CcS-7509	b-7546	2078	51.9367	5.34765	2023-12-31 00:06:36	383.63	0

transaction 5 x

Output

#	Time	Action	Message	Duration / Fetch
6	10:38:23	SELECT company.id, company.company_name, AVG(transaction.amount) AS medi...	1 row(s) returned	0.359 sec / 0.000 sec
7	10:39:42	SELECT * FROM transaction WHERE EXISTS (SELECT company.id FROM comp...	1000 row(s) returned	0.015 sec / 0.000 sec

- Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions:

```
77 • SELECT DISTINCT company.company_name
78 FROM company
79 JOIN transaction
80 ON company.id = transaction.company_id
81 WHERE transaction.amount > (SELECT AVG(transaction.amount)
82                             FROM transaction
83                             WHERE transaction.declined = 0);
84
```

Result Grid

company_name
Eget Tincidunt Dui Institute
Neque Tellus Imperdiet Corp.
Aliquam Iaculis Lacus Corp.
Lorem Ipsum Dolor Corp.
Nulla Integer Vulputate Corp.
Auctor Mauris Corp.
Turbo Faucibus Ltd.

Result 30 x

Output

#	Time	Action	Message	Duration / Fetch
54	13:02:49	SELECT DISTINCT company.company_name FROM company JOIN transaction O...	Error Code: 1054. Unknown column 'transaction.declined' in 'where clause'	0.000 sec
55	13:02:58	SELECT DISTINCT company.company_name FROM company JOIN transaction O...	100 row(s) returned	0.203 sec / 0.000 sec

- *Eliminarán del sistema las empresas que no tienen transacciones registradas, entrega el llistat d'aquestes empreses:*

```

85 • SELECT company.company_name
86 FROM company
87 WHERE NOT EXISTS (SELECT transaction.company_id
88 FROM transaction);

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

company_name

company 7 x | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
9	10:42:17	SELECT company.company_name FROM company WHERE company.id IN (SELE...	Error Code: 1111. Invalid use of group function	0.016 sec
10	10:45:22	SELECT company.company_name FROM company WHERE NOT EXISTS (SELE...	0 row(s) returned	0.000 sec / 0.000 sec

No nos da ningún resultado porque no hay nada que eliminar ya que todas las empresas tienen transacciones registradas. En la tabla **company** hay 100 registros y realizo un SELECT DISTINCT para cerciorarme que son también 100 compañías únicas en la tabla **transaction** :

```

58 • SELECT DISTINCT transaction.company_id
59 FROM transaction;
60
61

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

company_id
b-2222
b-2226
b-2230
b-2234
b-2238
b-2242
b-2246

transaction 17 x | Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
33	11:15:12	SELECT DISTINCT company.id FROM company LIMIT 0, 1000	100 row(s) returned	0.016 sec / 0.000 sec
34	11:16:37	SELECT DISTINCT transaction.company_id FROM transaction LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingessis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

Diseño y creo la tabla:

```
105 CREATE TABLE credit_card (  
106     id VARCHAR(15) NOT NULL,  
107     iban VARCHAR(34),  
108     pan VARCHAR(20),  
109     pin VARCHAR(4),  
110     cvv VARCHAR(3),  
111     expiring_date VARCHAR(8),  
112     PRIMARY KEY (id))  
113     ENGINE= InnoDB;
```

Output

#	Time	Action	Message	Duration / Fetch
100201	13:42:29	drop tables users	0 row(s) affected	0.031 sec
100202	13:42:51	CREATE TABLE credit_card (id VARCHAR(15) NOT NULL, iban VARCHAR(...	0 row(s) affected	0.046 sec

Insertamos los datos ejecutando el programa **N1-Ex.4 datos introducir credit.sql**:

Muestro la tabla con los registros cargados:

1 • `SELECT * FROM transactions.credit_card;`

id	iban	pan	pin	cvv	expiring_date
CcS-4857	XX4857591835292505850771	2314242385113924	1819	467	09/27/25
CcS-4858	XX8581768137002436094025	6582720299715533	3964	817	12/28/28
CcS-4859	XX7826930491423553609370	8861684536289642	4983	277	11/26/26
CcS-4860	XX5559590368835304645299	2481155515498459	6876	661	07/27/27
CcS-4861	XX2035182877195191627307	1308930301149557	5710	398	04/25/26
CcS-4862	XX4774721462463645409758	6715617009807829	4042	174	11/27/26
CcS-4863	XX1476829664245046207111	3140879819451394	5969	449	12/27/29
CcS-4864	XX8380298893385731196159	5793672133649114	8481	139	02/28/26
CcS-4865	XX7085078596101025280599	5101552687251312	7847	903	11/25/28
CcS-4866	XX4792859188206596406839	8080768801072613	9271	961	02/28/25
CcS-4867	XX6038298816319374853717	7761849537661098	4820	862	11/30/25

credit_card 1 x

Output

#	Time	Action	Message	Duration / Fetch
5043	11:32:49	INSERT INTO credit_card (id, iban, pan, pin, cvv, expiring_date) VALUES (CcS-95...	1 row(s) affected	0.032 sec
5044	11:35:09	SELECT * FROM transactions.credit_card LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.016 sec

Para poder definir la columna 'expiring_date' a formato DATE desactivamos el modo seguro temporalmente:

```
117 • SET SQL_SAFE_UPDATES = 0;
118
```

#	Time	Action	Message	Duration / Fetch
5028	10:49:44	ALTER TABLE credit_card DROP COLUMN expiring_date_temporal	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.016 sec
5029	10:53:38	SET SQL_SAFE_UPDATES = 0	0 row(s) affected	0.000 sec

Actualizamos la columna:

```
120 • UPDATE credit_card
121 SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%Y');
```

#	Time	Action	Message	Duration / Fetch
5032	10:55:20	UPDATE credit_card SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%Y')	5000 row(s) affected Rows matched: 5000 Changed: 5000 Warnings: 0	0.188 sec

Volvemos a activar el modo seguro:

```
124 • SET SQL_SAFE_UPDATES = 1;
125
```

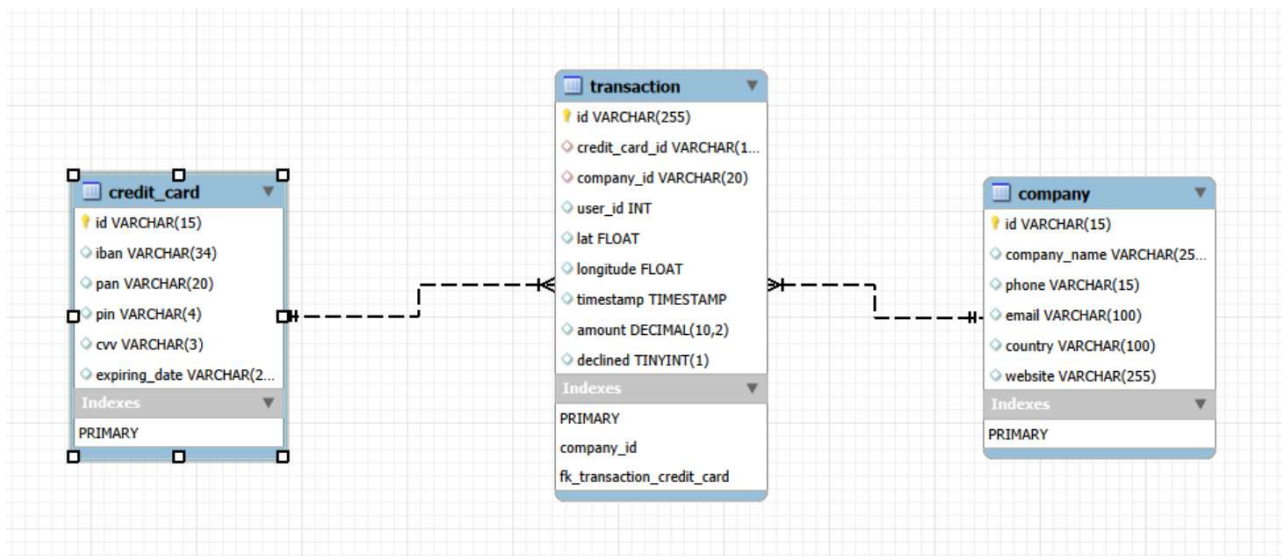
#	Time	Action	Message	Duration / Fetch
5034	10:56:09	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec

Modifico la tabla **transaction** con ALTER TABLE añadiendo restricción foreign key. Esta foreign key 'credit_card' tiene que corresponder a un 'id' que exista en **credit_card**:

```
126 -- Modificamos la tabla 'transaction' para añadir foreign key
127 • ALTER TABLE transaction
128 ADD CONSTRAINT fk_transaction_credit_card
129 FOREIGN KEY (credit_card_id)
130 REFERENCES credit_card(id);
131
```

#	Time	Action	Message	Duration / Fetch
5035	10:56:13	SET SQL_SAFE_UPDATES = 1	0 row(s) affected	0.000 sec
5036	11:05:38	ALTER TABLE transaction ADD CONSTRAINT fk_transaction_credit_card FOREIGN KEY (credit_card_id) REFERENCES credit_card(id)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.547 sec

Diagrama:



El diagrama muestra dos relaciones **1:N**. Por un lado, una **credit_card** puede tener **muchas transaction**, pero cada transacción pertenece a una única tarjeta, mediante 'credit_card_id' en la tabla **transaction**. Por otro lado, una **company** puede tener **muchas transaction**, mientras que cada transacción está asociada a una única empresa, mediante 'company_id'. Por tanto, **transaction** está en el lado **N** de ambas relaciones y contiene las foreign key que la relacionan con **credit_card** y **company**.

EXERCICI 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

Hago el check del registro:

```

140 • SELECT *
141 FROM credit_card
142 WHERE id = 'CcU-2938';
143
144
  
```

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	2022-10-30

credit_card 2 x

Output

#	Time	Action	Message	Duration / Fetch
5041	11:33:40	SELECT * FROM transactions.credit_card LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
5042	11:34:13	SELECT * FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Utilizo el UPDATE para modificar datos existentes en la tabla **credit_card**:

```
146 • UPDATE credit_card
147 SET iban = 'TR323456312213576817699999'
148 WHERE id = 'CcU-2938';
```

Output

#	Time	Action	Message	Duration / Fetch
5042	11:34:13	SELECT * FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
5043	11:36:07	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = '...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec

Compruebo que el cambio se haya realizado:

```
151 • SELECT*
152 FROM credit_card
153 WHERE id = 'CcU-2938';
154
155
```

Result Grid

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	2022-10-30
*	NULL	NULL	NULL	NULL	NULL

credit_card 3 x

Output

#	Time	Action	Message	Duration / Fetch
5043	11:36:07	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = '...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
5044	11:36:57	SELECT* FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 6

En la taula "transaction" ingresa una nova transacció amb la següent informació:

```
Id
10881D1D-5B23-A76C-55EF-C568E49A99DD

credit_card_id

CcU-9999

company_id

b-9999

user_id

9999

lat

829.999

longitude

-117.999

amount

111.11

declined

0
```

Insertamos en la tabla **company** el 'id' si no existe. 'id' es la primary key de **company** y la foreign key de **transaction**, por lo tanto es requisito imprescindible que exista el registro.

```

160 • INSERT INTO company (id)
161     SELECT 'b-9999'
162 WHERE NOT EXISTS (
163     SELECT company.id
164     FROM company
165     WHERE company.id = 'b-9999'
166 );
167

```

Output

#	Time	Action	Message	Duration / Fetch
5047	11:47:53	INSERT INTO company (id) SELECT b-9999 WHERE NOT EXISTS (SELECT c...	Error Code: 1054. Unknown column 'b' in field list'	0.000 sec
5048	11:48:01	INSERT INTO company (id) SELECT b-9999 WHERE NOT EXISTS (SELECT ...	1 row(s) affected Records: 1 Duplicates: 0 Warnings: 0	0.015 sec

Insertamos en la tabla **credit_card** el 'id' si no existe. 'id' es la primary key de **credit_card** y la foreign key de **transaction**, por lo tanto es requisito imprescindible que exista el registro.

```

168 • INSERT INTO credit_card (id)
169     SELECT 'CcU-9999'
170 WHERE NOT EXISTS (
171     SELECT credit_card.id
172     FROM credit_card
173     WHERE credit_card.id = 'CcU-9999'
174 );

```

Output

#	Time	Action	Message	Duration / Fetch
5048	11:48:01	INSERT INTO company (id) SELECT b-9999 WHERE NOT EXISTS (SELECT ...	1 row(s) affected Records: 1 Duplicates: 0 Warnings: 0	0.015 sec
5049	11:56:13	INSERT INTO credit_card (id) SELECT 'CcU-9999' WHERE NOT EXISTS (SEL...	1 row(s) affected Records: 1 Duplicates: 0 Warnings: 0	0.000 sec

Ahora ya podemos insertar los datos a la tabla **transaction**:

```

176 -- Insertamos los datos en la tabla transaction:
177 • INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
178     VALUES ('10881D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', '9999', 829.999, -117.999, 111.11, '0');
179
180

```

Output

#	Time	Action	Message	Duration / Fetch
5043	11:36:07	UPDATE credit_card SET iban = 'TR323456312213576817699999' WHERE id = '...	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.031 sec
5044	11:36:57	SELECT* FROM credit_card WHERE id = 'CcU-2938' LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec

Chequeamos el cambio:

The screenshot shows a database interface with a SQL query editor at the top containing lines 181 and 182:

```
181 • SELECT*
182 FROM transaction
```

Below the editor is a 'Result Grid' showing a table with 10 columns: id, credit_card_id, company_id, user_id, lat, longitude, timestamp, amount, declined, and an unnamed column. The first row has values: 108B1D1D-5B23-A76C-55EF-C568E49A99DD, CdU-9999, b-9999, 9999, 829.999, -117.999, NULL, 111.11, 0, and NULL. The second row is all NULL.

Below the result grid is an 'Output' pane showing an 'Action Output' log with two entries:

#	Time	Action	Message	Duration / Fetch
5054	12:12:09	INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, a...	Error Code: 1062. Duplicate entry '108B1D1D-5B23-A76C-55EF-C568E49A99DD' f...	0.000 sec
5055	12:12:25	SELECT* FROM transaction WHERE transaction.id = '108B1D1D-5B23-A76C-55E...	1 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recordar mostrar el canvi realitzat.

Utilizamos **drop column** para eliminar una columna existente en la tabla credit_card.

The screenshot shows a database interface with a SQL query editor containing lines 189 and 190:

```
189 -- Eliminamos columna:
190 • alter table credit_card drop column pan;
```

Below the editor is an 'Output' pane showing an 'Action Output' log with two entries:

#	Time	Action	Message	Duration / Fetch
5055	12:12:25	SELECT* FROM transaction WHERE transaction.id = '108B1D1D-5B23-A76C-55E...	1 row(s) returned	0.000 sec / 0.000 sec
5056	12:13:51	alter table credit_card drop column pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec

Con el INFORMATION_SCHEMA.COLUMNS comprobamos cuántas columnas tiene ahora **credit_card**:

The screenshot shows a database interface with a SQL query editor containing lines 194 to 197:

```
194 • SELECT COLUMN_NAME
195 FROM INFORMATION_SCHEMA.COLUMNS
196 WHERE TABLE_SCHEMA = 'transactions'
197 AND TABLE_NAME = 'credit_card';
```

Below the editor is a 'Result Grid' showing a table with 1 column: COLUMN_NAME. The rows list the columns: cvv, expiring_date, iban, id, and pin.

Below the result grid is an 'Output' pane showing an 'Action Output' log with two entries:

#	Time	Action	Message	Duration / Fetch
5060	12:20:54	SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE ...	5 row(s) returned	0.000 sec / 0.000 sec
5061	12:21:01	SELECT COLUMN_NAME FROM INFORMATION_SCHEMA.COLUMNS WHERE ...	5 row(s) returned	0.000 sec / 0.000 sec

Comprobamos el cambio en la tabla **credit_card**:

The screenshot shows a database interface with a SQL query editor at the top containing the query: `SELECT* FROM credit_card;`. Below the editor is a 'Result Grid' showing a table with 5 columns: `id`, `iban`, `pin`, `cvv`, and `expiring_date`. The table contains 5 rows of data. Below the result grid is an 'Output' section with a table showing the execution of two actions:

#	Time	Action	Message	Duration / Fetch
5056	12:13:51	alter table credit_card drop column pan	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.047 sec
5057	12:15:13	SELECT* FROM credit_card LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 8

Descarrega els arxius CSV que **trobaràs a l'apartat de recursos**:

- *american_users.csv*
- *european_users.csv*
- *companies.csv*
- *credit_cards.csv*
- *transactions.csv*

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

La taula de *products.csv* l'utilitzarem més endavant.

Creamos la base de datos **ONLINE_SALES**:

The screenshot shows a database interface with a SQL query editor at the top containing the query: `CREATE DATABASE IF NOT EXISTS online_sales;`. Below the editor is an 'Output' section with a table showing the execution of one action:

#	Time	Action	Message	Duration / Fetch
5052	12:11:27	CREATE DATABASE IF NOT EXISTS online_sales	1 row(s) affected	0.016 sec

Desde los archivos csv descargados, creamos una base de datos con cuatro tablas:

- **companies**: Datos de identificación de las compañías
- **transactions**: Tabla de hechos con los datos de las transacciones realizadas
- **credit_cards**: Datos de identificación de las diferentes tarjetas de crédito
- **users**: Uniendo la tabla *american-users* y *european_users* con datos de todos los usuarios registrados y cambiamos el nombre de la columna 'country' por 'region' para que los datos tengan más coherencia.

Creamos la tabla **companies**:

```
214 • CREATE TABLE IF NOT EXISTS companies (  
215     company_id VARCHAR(15) NOT NULL,  
216     company_name VARCHAR(255),  
217     phone VARCHAR(20),  
218     email VARCHAR(100),  
219     country VARCHAR(100),  
220     website VARCHAR(255),  
221     merchant_category VARCHAR(100),  
222     merchant_price_position VARCHAR(50),  
223     PRIMARY KEY (company_id)  
224 );  
225
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
15092	12:34:56	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(15) NOT ...	0 row(s) affected	0.031 sec

Creamos la tabla **users**:

```
227 • CREATE TABLE IF NOT EXISTS users (  
228     id INT NOT NULL,  
229     name VARCHAR(100),  
230     surname VARCHAR(100),  
231     phone VARCHAR(30),  
232     email VARCHAR(100),  
233     birth_date VARCHAR(50),  
234     region VARCHAR(100),  
235     city VARCHAR(100),  
236     postal_code VARCHAR(20),  
237     address VARCHAR(255),  
238     signup_date DATE,  
239     user_segment VARCHAR(100),  
240     income_band VARCHAR(50),  
241     PRIMARY KEY (id)  
242 );  
243
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
15100	12:53:29	CREATE TABLE IF NOT EXISTS users (id INT NOT NULL, name VARCHAR...	0 row(s) affected	0.078 sec

Creamos la tabla **credit_cards**:

```
246 CREATE TABLE IF NOT EXISTS credit_cards (  
247     id VARCHAR(15) NOT NULL,  
248     user_id INT,  
249     iban VARCHAR(34),  
250     pan VARCHAR(20),  
251     pin VARCHAR(4),  
252     cvv VARCHAR(3),  
253     track1 VARCHAR(255),  
254     track2 VARCHAR(255),  
255     expiring_date VARCHAR(8),  
256     card_type VARCHAR(50),  
257     card_renewal_flag BOOLEAN,  
258     PRIMARY KEY (id),  
259     FOREIGN KEY (user_id) REFERENCES users(id)  
260 );  
261
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 15100	12:53:29	CREATE TABLE IF NOT EXISTS users (id INT NOT NULL, name VARCHAR...	0 row(s) affected	0.078 sec
✓ 15101	12:53:34	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) NOT NULL, ...	0 row(s) affected	0.078 sec

Y finalmente la tabla **transactions**:

```
264 CREATE TABLE IF NOT EXISTS transactions (  
265     id VARCHAR(100) NOT NULL,  
266     card_id VARCHAR(15),  
267     business_id VARCHAR(15),  
268     timestamp DATETIME,  
269     amount DECIMAL (10,2),  
270     declined BOOLEAN,  
271     product_ids VARCHAR(150),  
272     user_id INT,  
273     lat FLOAT,  
274     longitude FLOAT,  
275     discount_amount DECIMAL (10,2),  
276     tax_amount DECIMAL (10,2),  
277     shipping_amount DECIMAL (10,2),  
278     channel VARCHAR(50),  
279     campaign_id VARCHAR(20),  
280     device_type VARCHAR(50),  
281     is_international BOOLEAN,  
282     decline_reason VARCHAR(20),  
283     distance_km DECIMAL(10,2),  
284     PRIMARY KEY (id),  
285     FOREIGN KEY (card_id) REFERENCES credit_cards(id),  
286     FOREIGN KEY (business_id) REFERENCES companies(company_id),  
287     FOREIGN KEY (user_id) REFERENCES users(id)  
288 );
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 15101	12:53:34	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(15) NOT NULL, ...	0 row(s) affected	0.078 sec
✓ 15102	12:55:04	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(100) NOT NULL, car...	0 row(s) affected	0.109 sec

Preguntamos a MySQL la ruta segura para guardar los archivos y poder cargarlos:

The screenshot shows the MySQL Workbench interface. The command editor at the top contains the query: `SHOW VARIABLES LIKE 'secure_file_priv';`. Below the command editor, the 'Result Grid' shows a single row with the variable name 'secure_file_priv' and its value 'C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\'. The 'Output' pane at the bottom shows the execution log with two entries: a successful 'CREATE TABLE' statement and the current 'SHOW VARIABLES' query which returned 1 row.

Variable_name	Value
secure_file_priv	C:\ProgramData\MySQL\MySQL Server 8.0\Uploads\

Subimos los archivos desde la ruta segura:

The screenshot shows a Windows File Explorer window titled 'Uploads'. The address bar shows the path: 'Disco local (C:) > ProgramData > MySQL > MySQL Server 8.0 > Uploads'. The main pane displays a list of files:

Nombre	Fecha de modificación	Tipo	Tamaño
N1-Ex.8_companies	09/09/2026 13:10	Hoja de cálculo de...	13 KB
N1-Ex.8_credit_cards	09/09/2026 13:09	Hoja de cálculo de...	851 KB
N1-Ex.8_transactions	12/09/2026 19:46	Hoja de cálculo de...	18.077 KB
N1-Ex.8_users	17/09/2026 12:52	Hoja de cálculo de...	719 KB

Cargamos el archivo N1-Ex.8__companies.csv a la tabla **companies**:

The screenshot shows the MySQL Workbench command editor with the following SQL command: `LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_companies.csv' INTO TABLE companies FIELDS TERMINATED BY ',' IGNORE 1 ROWS;`. The 'Output' pane shows the execution log with two entries: the previous 'SHOW VARIABLES' query and the current 'LOAD DATA INFILE' command, which successfully loaded 100 rows.

#	Time	Action	Message	Duration / Fetch
15103	12:56:04	SHOW VARIABLES LIKE 'secure_file_priv'	1 row(s) returned	0.000 sec / 0.000 sec
15104	12:58:41	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....'	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec

Cargamos el archivo N1-Ex.8__users.csv a la tabla **users**:

The screenshot shows the MySQL Workbench command editor with the following SQL command: `LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8_users.csv' INTO TABLE users FIELDS TERMINATED BY ',' ENCLOSED BY '''' IGNORE 1 ROWS;`. The 'Output' pane shows the execution log with two entries: the previous 'LOAD DATA INFILE' command and the current one, which successfully loaded 5000 rows.

#	Time	Action	Message	Duration / Fetch
15104	12:58:41	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....'	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.062 sec
15105	13:01:03	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....'	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.359 sec

Cargamos el archivo N1-Ex.8__credit_cards.csv a la tabla **credit_cards**:

```
307 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__credit_cards.csv'
308 INTO TABLE credit_cards
309 FIELDS TERMINATED BY ','
310 IGNORE 1 ROWS;
311
```

Output :

#	Time	Action	Message	Duration / Fetch
15105	13:01:03	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.359 sec
15106	13:02:48	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....	5000 row(s) affected Records: 5000 Deleted: 0 Skipped: 0 Warnings: 0	0.328 sec

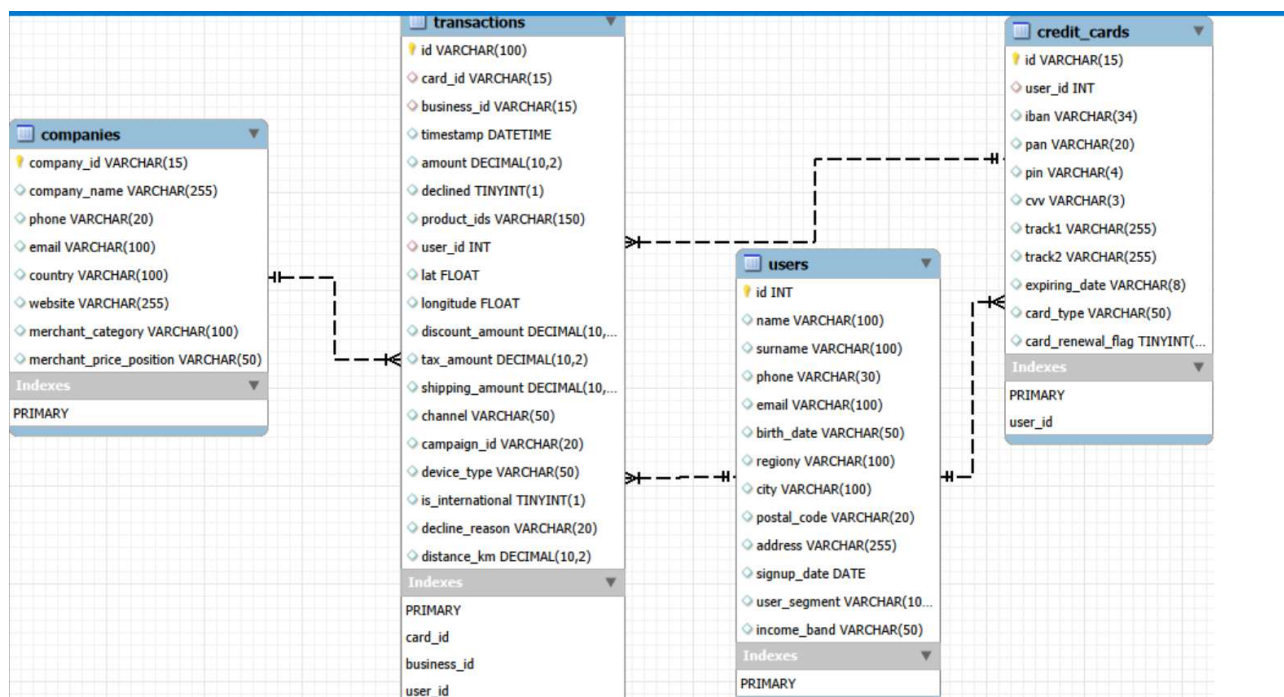
Cargamos el archivo N1-Ex.8__transactions.csv a la tabla **transactions**:

```
313 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__transactions.csv'
314 INTO TABLE transactions
315 FIELDS TERMINATED BY ','
316 IGNORE 1 ROWS;
317
```

Output :

#	Time	Action	Message	Duration / Fetch
15108	13:03:53	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....	100000 row(s) affected Records: 100000 Deleted: 0 Skipped: 0 Warnings: 0	8.094 sec

Diagrama:



El diagrama representa un **modelo en estrella**, donde la tabla central de **transactions** es la **tabla de hechos**, ya que contiene las operaciones y medidas como amount, discount_amount, tax_amount, shipping_amount, etc. Las tablas **companies**, **credit_cards** y **users** actúan como tablas relacionadas con los hechos:

- La relación entre **companies** y **transactions** es **1:N**: una empresa puede tener **muchas transacciones**, pero cada transacción pertenece a **una sola empresa**, mediante business_id.
- Entre **credit_cards** y **transactions** también hay una **relación 1:N**: una tarjeta puede utilizarse en **muchas transacciones**, mientras que cada transacción corresponde a **una sola tarjeta**, mediante card_id.
- Entre **users** y **transactions** hay **igualmente una relación 1:N**: un usuario puede realizar **muchas transacciones**, pero cada transacción pertenece a **un solo usuario**, mediante user_id.
- Además, existe una relación entre **users** y **credit_cards** de **1:N**: un usuario puede tener **muchas tarjetas**, mientras que cada tarjeta pertenece a **un solo usuario**, mediante credit_cards.user_id.
- Por tanto, en las relaciones con transactions, las tablas **companies**, **credit_cards** y **users** están en el lado 1, y **transactions** está en el lado N (muchos).

EXERCICI 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

```

322 • SELECT users.id, users.name, users.surname
323 FROM users
324 WHERE users.id IN (SELECT superior_ochenta.user_id
325                     FROM ( SELECT transactions.user_id,COUNT(*) as numero_transacciones
326                           FROM transactions
327                           WHERE transactions.declined = 0
328                           GROUP BY transactions.user_id
329                           HAVING numero_transacciones > 80) as superior_ochenta);
330

```

id	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bnyr	Astuw
454	Sfzzoh	Xgvfridxs
NULL	NULL	NULL

users 20 x

Output

#	Time	Action	Message	Duration / Fetch
15118	13:19:55	SELECT users.id, users.name, users.surname FROM users WHERE users.id IN (S...	4 row(s) returned	0.312 sec / 0.000 sec
15119	13:21:06	SELECT users.id, users.name, users.surname FROM users WHERE users.id IN (S...	4 row(s) returned	0.266 sec / 0.000 sec

EXERCICI 10

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

```
333 • SELECT companies.company_name, credit_cards.iban, ROUND(AVG(transactions.amount),2) as media
334 FROM transactions
335 JOIN companies
336 ON transactions.business_id = companies.company_id
337 JOIN credit_cards
338 ON transactions.card_id = credit_cards.id
339 WHERE companies.company_name = 'Donec Ltd'
340 AND transactions.declined = 0
341 GROUP BY credit_cards.iban
342 ORDER BY media DESC;
```

Result Grid

	company_name	iban	media
▶	Donec Ltd	US212768278745981809500233	708.14
	Donec Ltd	CA817443448587215122047864	690.81
	Donec Ltd	US157275989571099139868887	647.08
	Donec Ltd	US594988240619959562749388	638.82
	Donec Ltd	US186466262266569948042760	629.76

Result 22 x Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 15120	13:24:03	SELECT companies.company_name, credit_cards.iban, ROUND(AVG(transactions...	367 row(s) returned	0.015 sec / 0.000 sec
✓ 15121	13:24:14	SELECT companies.company_name, credit_cards.iban, ROUND(AVG(transactions...	367 row(s) returned	0.000 sec / 0.000 sec

NIVELL 2

EXERCICI 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

```
350 • SELECT DATE(timestamp), ROUND(SUM(amount),2) as suma
351 FROM transactions
352 WHERE transactions.declined = 0
353 GROUP BY DATE(timestamp)
354 ORDER BY suma DESC
355 LIMIT 5;
356
```

DATE(timestamp)	suma
2023-02-20	15728.40
2019-11-18	15447.89
2022-12-13	14944.00
2019-03-18	14896.47
2017-12-20	14367.96

Result 29 x

Output

#	Time	Action	Message	Duration / Fetch
15128	13:28:36	SELECT DATE(timestamp), ROUND(SUM(amount),2) as suma FROM transactions ...	Error Code: 1055. Expression #1 of ORDER BY clause is not in GROUP BY clause ...	0.000 sec
15129	13:28:49	SELECT DATE(timestamp), ROUND(SUM(amount),2) as suma FROM transactions ...	5 row(s) returned	0.125 sec / 0.000 sec

EXERCICI 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

```
360 • SELECT companies.company_name, companies.phone, companies.country, transactions.timestamp, transactions.amount
361 FROM companies
362 JOIN transactions
363 ON companies.company_id = transactions.business_id
364 WHERE transactions.amount BETWEEN 350 AND 400
365 AND DATE(timestamp) IN ('2015-04-29', '2018-07-20', '2024-03-13')
366 AND transactions.declined = 0
367 ORDER BY amount DESC;
368
```

company_name	phone	country	timestamp	amount
Fringilla LLC	+1 150 784 1716	New Zealand	2015-04-29 06:18:25	390.33
Auctor Mauris Vel LLP	+32 133 447 8239	United States	2024-03-13 01:23:09	383.58
Mauris Institute	+75 473 619 9472	Sweden	2015-04-29 21:24:49	369.20
Pede Cum Ltd	+26 625 798 6126	Norway	2018-07-20 14:57:32	358.64
Netus Et Malesuada Ltd	+52 818 402 1638	Netherlands	2024-03-13 13:20:45	352.21

Result 2 x

Output

#	Time	Action	Message	Duration / Fetch
3	21:31:50	SELECT DATE(timestamp), ROUND(SUM(amount),2) as suma FROM transactions ...	5 row(s) returned	0.109 sec / 0.000 sec
4	21:32:04	SELECT companies.company_name, companies.phone, companies.country, transa...	6 row(s) returned	0.093 sec / 0.000 sec

EXERCICI 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

```
374 • SELECT transactions.business_id, companies.company_name, COUNT(*) as numero_transacciones, CASE
375     WHEN COUNT(*) >= 400 THEN '400 o més'
376     ELSE 'Menys de 400'
377     END AS categoria
378 FROM transactions
379 JOIN companies
380 ON transactions.business_id = companies.company_id
381 WHERE transactions.declined = 0
382 GROUP BY transactions.business_id;
```

Result Grid

business_id	company_name	numero_transacciones	categoria
b-2222	Ac Fermentum Incorporated	2387	400 o més
b-2226	Magna A Neque Industries	408	400 o més
b-2230	Fusce Corp.	444	400 o més
b-2234	Convallis In Incorporated	1501	400 o més
b-2238	Ante Iaculis Nec Foundation	471	400 o més

Result 4 x

Output

#	Time	Action	Message	Duration / Fetch
5	21:36:08	SELECT transactions.business_id, companies.company_name, COUNT(*) as nume...	100 row(s) returned	0.094 sec / 0.000 sec
6	21:38:29	SELECT transactions.business_id, companies.company_name, COUNT(*) as nume...	100 row(s) returned	0.281 sec / 0.000 sec

EXERCICI 4

Comprobamos que el registro exista:

```
389 • SELECT *
390 FROM transactions
391 WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
392
```

Result Grid

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_a
000447FE-B650-4DCF-85DE-C7ED0EE1CAAD	Cc5-5019	b-2370	2016-12-21 20:07:18	172.56	0	66, 69, 87	438	41.5972	12.2218	19.92	29.86
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

transactions 5 x

Output

#	Time	Action	Message	Duration / Fetch
7	21:41:43	SELECT * FROM transaction WHERE transaction.id = '000447FE-B650-4DCF-85D...	Error Code: 1146. Table 'online_sales.transaction' doesn't exist	0.000 sec
8	21:42:05	SELECT * FROM transactions WHERE transactions.id = '000447FE-B650-4DCF-85...	1 row(s) returned	0.015 sec / 0.000 sec

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

```
394 • DELETE FROM transactions
395 WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
396
```

Output

#	Time	Action	Message	Duration / Fetch
8	21:42:05	SELECT * FROM transactions WHERE transactions.id = '000447FE-B650-4DCF-85...	1 row(s) returned	0.015 sec / 0.000 sec
9	21:43:15	DELETE FROM transactions WHERE ID = '000447FE-B650-4DCF-85DE-C7ED0E...	1 row(s) affected	0.032 sec

Comprobamos que se haya eliminado el registro:

```
398 • SELECT *
399 FROM transactions
400 WHERE transactions.id = '000447FE-B650-4DCF-85DE-C7ED0EE1CAAD';
401
```

Result Grid

id	card_id	business_id	timestamp	amount	declined	product_ids	user_id	lat	longitude	discount_amount	tax_amount	shipping_amount	channel	campaign_id

transactions 7 x

Output

#	Time	Action	Message	Duration / Fetch
10	21:45:43	SELECT ROW_COUNT() LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
11	21:46:39	SELECT * FROM transactions WHERE transactions.id = '000447FE-B650-4DCF-85...	0 row(s) returned	0.000 sec / 0.000 sec

EXERCICI 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

Creamos la vista:

```
411 SELECT companies.company_name, companies.phone, companies.country, AVG(transactions.amount) as Media
412 FROM companies
413 JOIN transactions
414 ON companies.company_id = transactions.business_id
415 GROUP BY companies.company_name, companies.phone, companies.country
416 ORDER BY Media DESC;
417
418
```

Output

#	Time	Action	Message	Duration / Fetch
13	21:49:17	CREATE VIEW VistaMarketing AS SELECT companies.company_name, companie...	0 row(s) affected	0.016 sec

Comprobamos la VistaMarketing:

```
419 • SELECT *
420 FROM online_sales.vistamarketing;
421
```

company_name	phone	country	Media
Ac Fermentum Incorporated	+64 601 244 6320	Germany	330.769338
Ut Semper Foundation	+10 283 500 6449	Sweden	307.372945
Ordi Adipiscing Limited	+60 897 390 5820	United Kingdom	306.265648
Dolor Vitae Limited	+6 421 567 9195	France	305.952745
Nunc In Foundation	+13 267 741 2619	Italy	305.396141

vistamarketing 8 x Read On

Output

#	Time	Action	Message	Duration / Fetch
✓ 15	21:50:33	SELECT * FROM online_sales.vistamarketing LIMIT 0, 1000	100 row(s) returned	0.407 sec / 0.000 sec
✓ 16	21:51:18	SELECT * FROM online_sales.vistamarketing LIMIT 0, 1000	100 row(s) returned	0.360 sec / 0.000 sec

NIVELL 3

EXERCICI 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

🔗 *Quantes targetes estan actives?*

Primero creamos la tabla desde una consulta utilizando el CASE WHEN para categorizar columnas y después el ROW_NUMBER() OVER (PARTITION BY....) para darles un número de fila a los registros por 'card_id' en orden descendiente, finalmente con el WHERE podemos quedarnos solamente con las tres últimas transacciones:

```
431 SELECT transacciones.card_id, CASE
432     WHEN SUM(transacciones.declined) = 3 THEN 'Inactiu'
433     ELSE 'Actiu'
434     END AS estado
435 FROM (SELECT card_id, declined, ROW_NUMBER() OVER (PARTITION BY card_id
436     ORDER BY timestamp DESC) as numero_fila
437     FROM transactions) as transacciones
438 WHERE transacciones.numero_fila IN (1,2,3)
439 GROUP BY card_id;
440
```

Output

#	Time	Action	Message	Duration / Fetch
✓ 16	21:51:18	SELECT * FROM online_sales.vistamarketing LIMIT 0, 1000	100 row(s) returned	0.360 sec / 0.000 sec
✓ 17	21:53:35	CREATE TABLE IF NOT EXISTS estado_targetes as SELECT transacciones.card...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.422 sec

Comprobamos la tabla:

```
442 • SELECT*
443 FROM estado_targetas;
444
```

Result Grid

card_id	estado
CcS-4857	Actiu
CcS-4858	Actiu
CcS-4859	Actiu
CcS-4860	Actiu
CcS-4861	Actiu

estado_targetas 9 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 17	21:53:35	CREATE TABLE IF NOT EXISTS estado_targetas as SELECT transacciones.card...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.422 sec
✓ 18	21:55:36	SELECT* FROM estado_targetas LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

Realizamos la consulta:

```
446 • SELECT COUNT(card_id)
447 FROM estado_targetas
448 WHERE estado = 'Actiu';
449
```

Result Grid

COUNT(card_id)
5000

Result 10 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
✓ 18	21:55:36	SELECT* FROM estado_targetas LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
✓ 19	21:57:33	SELECT COUNT(card_id) FROM estado_targetas WHERE estado = 'Actiu' LIMIT ...	1 row(s) returned	0.016 sec / 0.000 sec

En la tabla **credit_cards** tenemos 5000 id únicos y la consulta sobre la tabla **transactions** nos devuelve también 5000 uds de tarjetas activas, por tanto todas las tarjetas registradas en **credit_cards** han realizado las últimas tres transacciones con una aceptada como mínimo.

EXERCICI 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

🔗 *Necessitem conèixer el nombre de vegades que s'ha venut cada producte.*

Primero creamos la estructura de la tabla **products**:

```
462 • CREATE TABLE IF NOT EXISTS products (  
463     id VARCHAR (10) NOT NULL,  
464     product_name VARCHAR (100),  
465     price DECIMAL (10,2),  
466     colour VARCHAR(20),  
467     weight DECIMAL (10,1),  
468     warehouse_id VARCHAR(50),  
469     category VARCHAR(50),  
470     brand VARCHAR(50),  
471     cost DECIMAL (10,2),  
472     launch_date DATE,  
473     PRIMARY KEY (id));  
474
```

Output

#	Time	Action	Message	Duration / Fetch
20	22:06:25	USE online_sales	0 row(s) affected	0.016 sec
21	22:07:48	CREATE TABLE IF NOT EXISTS products (id VARCHAR (10) NOT NULL, produc...	0 row(s) affected	0.046 sec

Cargamos el archivo N1-Ex.8__products con los registros para la tabla **products**:

Nos informa de un error en las fechas porque al final tiene un retorno, pero carga los datos correctamente:

```
476 • LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex.8__products.csv'  
477 INTO TABLE products  
478 FIELDS TERMINATED BY ','  
479 IGNORE 1 ROWS;  
480
```

Output

#	Time	Action	Message	Duration / Fetch
21	22:07:48	CREATE TABLE IF NOT EXISTS products (id VARCHAR (10) NOT NULL, produc...	0 row(s) affected	0.046 sec
22	22:10:02	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....	100 row(s) affected, 100 warning(s): 4095 Delimiter '~' in position 10 in datetime val...	0.015 sec

Comprobamos los registros cargados:

```
482 • SELECT *
483 FROM products;
```

Result Grid

	id	product_name	price	colour	weight	warehouse_id	category	brand	cost	launch_date
▶	1	Direwolf Stannis	161.11	#7c7c7c	1.0	WH-4	Sports	Runway Pro	63.84	2014-11-01
	10	Karstark Dorne	119.52	#d8d8d15	2.4	WH-5	Food	FreshBox	89.07	2013-10-04
	100	south duel	40.43	#d8d8d105	3.0	WH-95	Electronics	PixelForge	24.95	2013-01-05
	11	Karstark Dorne	49.70	#d8d8d16	2.7	WH-6	Electronics	Nexa	34.45	2014-09-01
	12	duel Direwolf	181.60	#d8d8d17	2.1	WH-7	Gaming	Questbyte	99.46	2013-10-11

products 11 x

Output

Action Output

#	Time	Action	Message	Duration / Fetch
22	22-10-02	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/N1-Ex....'	100 row(s) affected, 100 warning(s): 4095 Delimiter '~' in position 10 in datetime val...	0.015 sec
23	22-11-21	SELECT * FROM products LIMIT 0, 1000	100 row(s) returned	0.000 sec / 0.000 sec

Ahora creamos una tabla **transaction_products** para poder relacionar la tabla **products** con **transactions**, para ello desde una consulta seleccionamos el contenido de la tabla **transactions** de la columna 'product_ids' dónde aparecen varios 'id' de producto, separarlos y convertimos en filas para poder relacionarlos con el 'id' de la nueva tabla **products**.

Lo hacemos desde una consulta utilizando la función CONCAT para colocar corchetes al principio y al final del texto original para convertirlo en una lista numérica en formato JSON.

Con la función JSON_TABLE se recorre la lista posición por posición con el comando \$[*]. y con la instrucción PATH '\$', extrae el número de la posición actual, lo define como un entero (INT) y lo guarda en la columna product_id dentro de una tabla temporal llamada **productos_separados**, que se fusiona mediante un JOIN con la tabla **transactions** original para que nos de el resultado que queremos.

```
486 • CREATE TABLE IF NOT EXISTS transaction_products AS
487 SELECT transactions.id, productos_separados.product_id
488 FROM transactions
489 JOIN JSON_TABLE(
490     CONCAT('[', transactions.product_ids, ']'),
491     '$[*]' COLUMNS (
492         product_id INT PATH '$'
493     )
494 ) AS productos_separados;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
25	22-27-40	DROP TABLE transaction_products	0 row(s) affected	0.032 sec
26	22-28-00	CREATE TABLE IF NOT EXISTS transaction_products AS SELECT transactions.id...	253388 row(s) affected Records: 253388 Duplicates: 0 Warnings: 0	2.797 sec

Comprobamos la tabla generada:

```
498 • SELECT*
499 FROM transaction_products;
500
```

id	product_id
00043A49-2949-494B-A5DD-A5BAE3BB19DD	16
00043A49-2949-494B-A5DD-A5BAE3BB19DD	26
00043A49-2949-494B-A5DD-A5BAE3BB19DD	97
00043A49-2949-494B-A5DD-A5BAE3BB19DD	87
00045D6B-ED2E-4F2F-8186-CEE074D875D0	30

transaction_products 13 x

Output

#	Time	Action	Message	Duration / Fetch
✓ 28	22:33:51	SELECT* FROM transaction_products LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec
✓ 29	22:34:22	SELECT* FROM transaction_products LIMIT 0, 1000	1000 row(s) returned	0.000 sec / 0.000 sec

Y finalmente hacemos la consulta que nos pide el ejercicio con un JOIN a la tabla que hemos creado **transaction_products** para extraer las veces que se ha vendido cada producto:

🔗 Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

```
502 • SELECT
503     products.id,
504     products.product_name,
505     COUNT(transaction_products.id) AS veces_vendido
506 FROM transaction_products
507 JOIN products
508     ON transaction_products.product_id = products.id
509 GROUP BY
510     products.id,
511     products.product_name;
```

id	product_name	veces_vendido
16	the duel warden	2608
26	Stark Karstark	2536
97	jinn Winterfell	2536
87	sith Jade	2597
30	Karstark warden	2468

Result 15 x

Output

#	Time	Action	Message	Duration / Fetch
✓ 33	22:37:23	SELECT products.id, products.product_name, COUNT(transaction_products.id) AS veces_vendido	100 row(s) returned	0.422 sec / 0.000 sec
✓ 34	22:37:25	SELECT products.id, products.product_name, COUNT(transaction_products.id) AS veces_vendido	100 row(s) returned	0.422 sec / 0.000 sec

Diagrama final:

